LAPORAN PRATIKUM STRUKTUR DATA ALGORITMA

Dibuat oleh:

Nama	**: Muhammad Nazar Ikramullah**
Kelas	**: TI-1C**
Mata kuliah	**: Struktur Data Algoritma**
Prodi	**: Teknik Informatika**
Dosen pengampu	**: Muhammad Reza Zulman S.ST.,M.Sc**

PROGRAM STUDI TEKNIK INFORMATIKA

POLITEKNIK NEGERI LHOKSEUMAWE

2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini semakin pesat dan berpengaruh besar dalam berbagai bidang, terutama dalam dunia pemrograman komputer. Pemrograman menjadi keterampilan penting karena digunakan untuk membuat aplikasi dan sistem yang dapat membantu pekerjaan manusia menjadi lebih mudah.

Dalam pemrograman, terdapat beberapa konsep dasar yang harus dipahami, seperti *function* dan *array*. *Function* adalah sekumpulan perintah dalam program yang dapat digunakan kembali, sehingga membantu mengurangi penulisan kode yang berulang dan membuat program lebih rapi serta mudah dikelola.

Selain itu, *array* merupakan struktur data yang digunakan untuk menyimpan banyak nilai dalam satu variabel. Dengan menggunakan *array*, data dapat diatur dengan lebih terstruktur dan mudah diakses, terutama saat mengolah data dalam jumlah banyak.

Pemahaman tentang *function* dan *array* sangat penting karena kedua konsep ini sering digunakan dalam berbagai bahasa pemrograman seperti JavaScript, PHP, Python, dan Java. Oleh karena itu, pembahasan dalam tugas ini bertujuan untuk memahami konsep dasar serta penerapannya agar dapat membuat program yang lebih efektif dan terstruktur.

1.2 Tujuan

Tujuan dari penyusunan tugas ini adalah sebagai berikut:

1. Memahami konsep dasar serta pengertian function dalam pemrograman.
2. Mengetahui peran dan kegunaan function agar program dapat disusun dengan lebih rapi dan efisien.
3. Memahami konsep dasar array sebagai salah satu struktur data dalam pemrograman.
4. Mengetahui cara memanfaatkan array untuk menyimpan dan mengelola data secara sistematis.
5. Mengembangkan kemampuan dasar pemrograman, khususnya dalam penggunaan function dan array.
6. Memenuhi tugas yang diberikan pada mata kuliah pemrograman dasar.

1.3 Manfaat

Manfaat dari penulisan tugas ini adalah sebagai berikut:

1. **Bagi Penulis**
   Memberikan tambahan pengetahuan dan pemahaman mengenai function dan array, serta melatih kemampuan dalam membuat program yang lebih terstruktur.

2. **Bagi Pembaca**
   Menjadi sumber referensi yang dapat membantu dalam memahami konsep dasar function dan array dalam pemrograman.

3. **Bagi Dunia Pendidikan**
   Dapat dijadikan sebagai bahan pendukung dalam proses pembelajaran, khususnya pada materi pemrograman dasar.

4. **Bagi Pengembangan Program**
   Memberikan dasar pemahaman yang kuat dalam pembuatan program yang lebih kompleks di masa mendatang.

BAB II

PEMBAHASAN

2.1 Pengertian Function

Function merupakan sekumpulan instruksi atau blok kode dalam pemrograman yang dirancang untuk menjalankan suatu tugas tertentu. Function dapat digunakan kembali (reusable) kapan saja diperlukan, sehingga membantu mengurangi penulisan kode yang berulang. Dengan adanya function, program dapat disusun dengan lebih terstruktur, efisien, dan mudah dipahami.

Dalam penggunaannya, function umumnya memiliki beberapa komponen penting, yaitu nama function, parameter, dan nilai kembalian (return). Nama function berfungsi sebagai identitas untuk memanggilnya, parameter digunakan untuk menerima input atau data yang akan diproses, sedangkan return berfungsi untuk mengembalikan hasil dari proses tersebut. Namun, tidak semua function harus memiliki nilai kembalian, karena hal tersebut disesuaikan dengan kebutuhan program.

Penggunaan function sangat membantu dalam pengembangan perangkat lunak, karena program yang besar dapat dipecah menjadi bagian-bagian kecil yang lebih sederhana. Hal ini memudahkan programmer dalam proses pengembangan, perbaikan, maupun pengujian program. Selain itu, function juga meningkatkan keterbacaan kode (readability), karena setiap bagian program memiliki fungsi yang jelas.

Dalam proyek berskala besar, function sangat penting karena memungkinkan beberapa programmer bekerja pada bagian yang berbeda secara bersamaan tanpa saling mengganggu. Hampir semua bahasa pemrograman modern, seperti JavaScript, Python, Java, PHP, dan C++, menerapkan konsep function sebagai dasar dalam pembuatan program. Oleh karena itu, pemahaman mengenai function menjadi hal yang sangat penting, terutama bagi mahasiswa Teknik Informatika sebagai dasar untuk mempelajari materi pemrograman yang lebih lanjut.

2.2 Cara Kerja Function

Function bekerja dengan menjalankan serangkaian perintah yang telah ditentukan di dalamnya saat function tersebut dipanggil. Prosesnya dimulai dari pendefinisian function, kemudian pemanggilan, hingga menghasilkan output sesuai instruksi yang ada.

Pertama, function dibuat atau dideklarasikan dengan nama tertentu. Di dalamnya berisi kumpulan kode yang akan dijalankan ketika function digunakan. Function juga dapat memiliki parameter yang berfungsi untuk menerima data dari luar.

Kedua, function akan dieksekusi ketika dipanggil, baik dari program utama maupun dari function lain. Jika terdapat parameter, maka nilai yang diberikan akan diproses sesuai dengan perintah yang telah ditentukan di dalam function tersebut.

Ketiga, setelah proses selesai, function akan menghasilkan output. Output ini bisa berupa nilai yang dikembalikan menggunakan *return*, atau langsung ditampilkan tanpa mengembalikan nilai, tergantung pada jenis function yang digunakan.

Dengan mekanisme tersebut, function membantu menghindari penulisan kode yang berulang, sehingga program menjadi lebih efisien, terstruktur, dan mudah untuk dikembangkan maupun dipelihara.

2.3 Manfaat Function

Function memiliki berbagai manfaat dalam pemrograman yang membantu mempermudah proses pembuatan program agar lebih efisien dan terorganisir. Beberapa manfaat dari penggunaan function antara lain sebagai berikut:

1. **Mengurangi pengulangan kode (code repetition)**
   Dengan adanya function, penulisan kode yang sama tidak perlu dilakukan berulang-ulang. Cukup dibuat satu kali, lalu dapat digunakan kembali kapan saja sesuai kebutuhan.

2. **Membuat program lebih terstruktur dan mudah dibaca**
   Function memungkinkan program yang besar dipecah menjadi bagian-bagian kecil. Hal ini membuat alur program menjadi lebih jelas, rapi, dan lebih mudah dipahami.

3. **Mempermudah proses debugging**
   Karena kode dibagi ke dalam beberapa function, kesalahan dalam program dapat lebih cepat ditemukan dan diperbaiki tanpa harus memeriksa seluruh bagian program.

2.4 Pengertian Array

Array merupakan salah satu struktur data dalam pemrograman yang digunakan untuk menyimpan sekumpulan nilai dalam satu variabel. Data yang tersimpan di dalam array memiliki tipe yang sama dan disusun secara berurutan. Setiap elemen dalam array memiliki indeks yang berfungsi untuk mengakses data tersebut, yang umumnya dimulai dari angka 0.

Penggunaan array sangat membantu dalam mengelola banyak data tanpa harus membuat banyak variabel terpisah. Dengan array, data seperti daftar nama, nilai, atau informasi lainnya dapat disimpan dengan lebih rapi dan efisien. Selain itu, array juga memudahkan proses pengolahan data, seperti menampilkan, mencari, mengubah, maupun melakukan perhitungan menggunakan perulangan.

Array memiliki beberapa jenis, di antaranya array satu dimensi dan array multi dimensi. Array satu dimensi digunakan untuk menyimpan data dalam satu baris, sedangkan array multi dimensi biasanya digunakan untuk menyimpan data dalam bentuk tabel atau matriks.

Sebagian besar bahasa pemrograman modern, seperti JavaScript, Python, Java, PHP, dan C++, mendukung penggunaan array sebagai struktur data dasar. Oleh karena itu, pemahaman tentang array sangat penting karena menjadi dasar dalam pengolahan data dan pengembangan program yang lebih kompleks.

2.5 Cara Kerja Array

Array bekerja dengan cara menyimpan sekumpulan data dalam satu variabel yang terstruktur dan setiap data di dalamnya memiliki indeks sebagai penanda posisi. Indeks ini digunakan untuk mengakses, membaca, maupun mengubah data yang ada di dalam array dengan mudah dan cepat. Ketika array dibuat, komputer akan mengalokasikan tempat di memori secara berurutan untuk menyimpan setiap elemen data. Setiap elemen tersebut kemudian diberi nomor indeks yang dimulai dari 0 untuk elemen pertama, 1 untuk elemen kedua, dan seterusnya. Dengan sistem indeks ini, programmer dapat langsung mengakses data tertentu tanpa harus memeriksa satu per satu dari awal. Saat proses pengolahan data dilakukan, array biasanya digunakan bersama dengan perulangan (looping) untuk mengakses semua elemen di dalamnya secara otomatis. Hal ini membuat proses pengolahan data menjadi lebih efisien, terutama jika jumlah data yang disimpan sangat banyak. Array juga memungkinkan data untuk dimodifikasi, seperti menambah, mengubah, atau menghapus nilai tertentu berdasarkan indeksnya. Dengan cara kerja seperti ini, array sangat membantu dalam pengelolaan data yang bersifat berulang atau dalam jumlah besar, karena lebih praktis dibandingkan menggunakan banyak variabel terpisah. Secara keseluruhan, cara kerja array berfokus pada penyimpanan data yang terstruktur dalam satu variabel dengan akses berbasis indeks, sehingga memudahkan proses pengolahan data dalam pemrograman.

2.6 Manfaat Array

Array memiliki banyak manfaat dalam pemrograman yang membuat proses pengolahan data menjadi lebih mudah, cepat, dan efisien. Salah satu manfaat utama array adalah kemampuannya untuk menyimpan banyak data dalam satu variabel, sehingga programmer tidak perlu membuat banyak variabel terpisah untuk setiap data. Hal ini membuat kode program menjadi lebih sederhana dan mudah dikelola. Selain itu, array sangat membantu dalam pengolahan data dalam jumlah besar. Dengan menggunakan array, data dapat diakses secara sistematis menggunakan indeks, sehingga proses pencarian, pembacaan, dan perubahan data menjadi lebih cepat. Hal ini sangat berguna dalam aplikasi yang menangani banyak data seperti sistem nilai mahasiswa, data inventaris barang, atau daftar pengguna. Manfaat lainnya adalah array dapat digunakan bersama dengan perulangan (looping) untuk memproses seluruh data secara otomatis.

Bab III

Langkah Langkah Pratikum

1.Buat nama folder sesuai nama repository lalu select folder

2025573010054_Algoritma_Dan_Struktur_Data	08/04/2026 9:15	File folder

2.Masuk git bash dan masuk ke direktori repository yang sebelum nya yang sudah di clone

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (mai
)
$ cd 2025573010054_Algoritma_Dan_Struktur_Data
```

3.Kemudian buat kan folder baru yang Bernama pratikum-3 untuk kita simpan di dalam nya

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (mai
)
$ mkdir pratikum-3
```

4.Setelah itu inisialisasi ke npm init y

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main
)
$ npm init -y
```

5.Kemudian kita akan buat file dalam folder modul 3

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main
)
$ touch 01-function-dasar.js
```

6.Kemudian ketikkan kode nya didalam file tersebut

```
function sapa() {
  console.log("Halo! Selamat datang di praktikum JavaScript.");
}
sapa(); // memanggil function
sapa(); // bisa dipanggil berkali-kali
// --- 2. Function dengan parameter ---
function sapaNama(nama) {
  console.log(`Halo, ${nama}! Selamat belajar.`);
}
sapaNama("Budi");
sapaNama("Siti");
// --- 3. Function dengan beberapa parameter dan return value ---
function tambah(angka1, angka2) {
```

```
  const hasil = angka1 + angka2;
  return hasil; // mengembalikan nilai
}
const hasilPenjumlahan = tambah(10, 25);
console.log("10 + 25 =", hasilPenjumlahan);
console.log("7 + 13 =", tambah(7, 13)); // bisa langsung dipakai
// --- 4. Function dengan default parameter ---
// Jika argumen tidak diberikan, gunakan nilai default
function hitung(nilai, pengali = 2) {
  return nilai * pengali;
}
console.log(hitung(5)); // 10 (pengali pakai default: 2)
console.log(hitung(5, 3)); // 15 (pengali diisi: 3)
// --- 5. Scope variabel ---
const pesanGlobal = "Saya ada di mana saja"; // global scope
function cekScope() {
  const pesanLokal = "Saya hanya ada di dalam function ini"; // local scope
  console.log(pesanGlobal); // bisa akses global
  console.log(pesanLokal); // bisa akses lokal
}
cekScope();
console.log(pesanGlobal); // bisa
```

10.Lalu coba tekan terminal dan run kan,kalo berhasil gini tampilannya

```
Saya ada di mana saja
Saya hanya ada di dalam function ini
Saya ada di mana saja
```

11.Setelah itu buiatkan Latihan 1: function kalkulator

```
console.log("\n=== Function Kalkulator Lengkap ===");

// Function tambah (jika sudah ada sebelumnya, tidak perlu buat lagi)
function tambah(a, b) {
  return a + b;
}

// Function kurang
function kurang(a, b) {
  return a - b;
}
```

```
// Function kali
function kali(a, b) {
  return a * b;
}

// Function bagi dengan pengecekan nol
function bagi(a, b) {
  if (b === 0) {
    console.log("Error: tidak bisa membagi dengan nol");
    return null;
  }
  return a / b;
}

// Function kalkulator utama
function kalkulator(a, operator, b) {
  if (operator === "+") {
    return tambah(a, b);
  } else if (operator === "-") {
    return kurang(a, b);
  } else if (operator === "*") {
    return kali(a, b);
  } else if (operator === "/") {
    return bagi(a, b);
  } else {
    console.log("Operator tidak dikenal");
    return null;
  }
}

// Pengujian (minimal 5 kali pemanggilan)
console.log("10 + 5 =", kalkulator(10, "+", 5));
console.log("10 - 5 =", kalkulator(10, "-", 5));
console.log("10 * 5 =", kalkulator(10, "*", 5));
console.log("10 / 5 =", kalkulator(10, "/", 5));

// Uji pembagian dengan nol
console.log("10 / 0 =", kalkulator(10, "/", 0));
```

12.Setelah itu simpan dan jalan kan di terminal

```
=== Function Kalkulator Lengkap ===
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2
Error: tidak bisa membagi dengan nol
10 / 0 = null
PS C:\Users\LENOVO\2025573010054_Algoritma_Dan_Struktur_Data>
```

13.Setelah itu buat kan file didalam modul pratikum 3 dengan nama 02-Arrow callback.js

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main)
$ touch 02-arrow-callback.js
```

14.setelah itu didalm file nya ketik kode js nya

```
function kuadratBiasa(x) {
  return x * x;
}
// Cara baru (arrow function - ekuivalen)
const kuadrat = (x) => {
  return x * x;
};
// Arrow function ringkas (implicit return)
const kuadratRingkas = (x) => x * x;
console.log("=== Perbandingan Penulisan ===");
console.log("Biasa :", kuadratBiasa(5)); // 25
console.log("Arrow :", kuadrat(5)); // 25
console.log("Ringkas :", kuadratRingkas(5)); // 25
// --- 2. Arrow function dengan beberapa parameter ---
const luas = (panjang, lebar) => panjang * lebar;
const salam = (nama, waktu) => `Selamat ${waktu}, ${nama}!`;
console.log("Luas 4x6 :", luas(4, 6));
console.log(salam("Budi", "Pagi"));
// --- 3. Callback: function sebagai argumen ---
function lakukanOperasi(angka, operasiCallback) {
  console.log(`Angka awal: ${angka}`);
  const hasil = operasiCallback(angka);
  console.log(`Hasil setelah operasi: ${hasil}`);
}
console.log("\n=== Callback ===");

lakukanOperasi(7, kuadratRingkas); // kirim function sebagai argumen
```

```
lakukanOperasi(10, (x) => x + 100); // kirim arrow function langsung
lakukanOperasi(20, function (x) {
  // kirim function biasa langsung
  return x / 2;
});
// --- 4. setTimeout sebagai contoh callback nyata ---
console.log("\n=== setTimeout (Callback) ===");
console.log("Pesan 1: Sebelum timer");
setTimeout(() => {
  console.log("Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)");
}, 1000); // 1000 milidetik = 1 detik
console.log("Pesan 2: Setelah mendaftarkan timer");
```

15.Setelah itu simpan dulu, lalu coba jalan kan

```
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10
```

16.buatkan Latihan 2 papilen tranformasi data didalam file arraw.js

```
//Latihan-2

// Arrow function ke huruf besar
const keHurufBesar = (teks) => {
  return teks.toUpperCase();
};

// Arrow function tambah tanda seru
const tambahSeru = (teks) => {
  return teks + "!!!";
```

```
};

// Arrow function hitung jumlah kata
const hitungKata = (teks) => {
  return teks.split(" ").length;
};

// Function dengan callback
function prosesKalimat(kalimat, transformasiCallback) {
  const hasil = transformasiCallback(kalimat);
  console.log(hasil);
}

// Kalimat uji
const kalimat = "belajar javascript itu menyenangkan";

// Pengujian
prosesKalimat(kalimat, keHurufBesar);
// Output: BELAJAR JAVASCRIPT ITU MENYENANGKAN

prosesKalimat(kalimat, tambahSeru);
// Output: belajar javascript itu menyenangkan!!!

prosesKalimat(kalimat, hitungKata);
// Output: 4
```

17.Simpan kan dulu lalu coba jalankan

```
Biasa : 25
Arrow : 25
Ringkas : 25
Luas 4x6 : 24
Selamat Pagi, Budi!

=== Callback ===
Angka awal: 7
Hasil setelah operasi: 49
Angka awal: 10
Hasil setelah operasi: 110
Angka awal: 20
Hasil setelah operasi: 10

=== setTimeout (Callback) ===
Pesan 1: Sebelum timer
Pesan 2: Setelah mendaftarkan timer
Pesan 3: Ini dari dalam setTimeout (setelah 1 detik)
PS C:\Users\LENOVO\2025573010054_Algoritma_Dan_Struktur_Data> node "c:\Users\LENOVO\2025573010054_Algoritma_Dan_Struktur_Data\pratiku
m-3\tempCodeRunnerFile.js"
BELAJAR JAVASCRIPT ITU MENYENANGKAN
belajar javascript itu menyenangkan!!!
4
```

18.kemudian buat kan file 03-array-dasar.js didalam folder pratikum 3

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main)
$ touch 03-array-dasar.js
```

19.Setelah itu coba ketikkan kodenya

```
const mahasiswa = ["Budi", "Siti", "Ahmad", "Rina"];
const nilai = [85, 92, 78, 95, 88];

console.log("=== Array Awal ===");
console.log("Mahasiswa:", mahasiswa);
console.log("Nilai :", nilai);
console.log("Jumlah mahasiswa:", mahasiswa.length);
// --- 2. Mengakses elemen ---
console.log("\n=== Akses Elemen ===");
console.log("Elemen pertama :", mahasiswa[0]); // indeks 0
console.log("Elemen ketiga :", mahasiswa[2]); // indeks 2
console.log("Elemen terakhir:", mahasiswa[mahasiswa.length - 1]);
// --- 3. Mengubah elemen ---
mahasiswa[1] = "Siti Rahayu"; // ubah elemen indeks 1
console.log("\nSetelah diubah:", mahasiswa);
// --- 4. Menambah dan menghapus elemen ---
console.log("\n=== Manipulasi Array ===");
mahasiswa.push("Doni"); // tambah di akhir
console.log("Setelah push :", mahasiswa);
mahasiswa.unshift("Zahra"); // tambah di awal
console.log("Setelah unshift :", mahasiswa);
const dihapusAkhir = mahasiswa.pop(); // hapus dari akhir
console.log("Dihapus (pop) :", dihapusAkhir);
console.log("Setelah pop :", mahasiswa);
const dihapusAwal = mahasiswa.shift(); // hapus dari awal
console.log("Dihapus (shift) :", dihapusAwal);
console.log("Setelah shift :", mahasiswa);
// --- 5. Mencari elemen ---
console.log("\n=== Pencarian ===");
console.log("Indeks Ahmad :", mahasiswa.indexOf("Ahmad"));
console.log("Ada Rina? :", mahasiswa.includes("Rina"));
console.log("Ada Budi? :", mahasiswa.includes("Budi"));
```

20. Simpan kan dan coba jalankan

```
=== Array Awal ===
Mahasiswa: [ 'Budi', 'Siti', 'Ahmad', 'Rina' ]
Nilai : [ 85, 92, 78, 95, 88 ]
Jumlah mahasiswa: 4

=== Akses Elemen ===
Elemen pertama : Budi
Elemen ketiga : Ahmad
Elemen terakhir: Rina

Setelah diubah: [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Manipulasi Array ===
Setelah push : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Setelah unshift : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina', 'Doni' ]
Dihapus (pop) : Doni
Setelah pop : [ 'Zahra', 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]
Dihapus (shift) : Zahra
Setelah shift : [ 'Budi', 'Siti Rahayu', 'Ahmad', 'Rina' ]

=== Pencarian ===
Indeks Ahmad : 2
Ada Rina? : true
Ada Budi? : true

Nilai asli : [ 85, 92, 78, 95, 88 ]
Slice [1,4] : [ 92, 78, 95 ]
```

21.Kemudian diikuti dengan latihan 3 yaitu menajemen daftar belanja didalam file array -3

```
// latihan-3

// 2. Membuat array daftarBelanja
let daftarBelanja = ["Beras", "Gula", "Minyak", "Telur", "Mie"];

// 3. Menampilkan isi array dengan nomor urut
console.log("Daftar Belanja:");
for (let i = 0; i < daftarBelanja.length; i++) {
  console.log(i + 1 + ". " + daftarBelanja[i]);
}

// 4. Menambahkan 2 item baru
daftarBelanja.push("Susu");
daftarBelanja.push("Kopi");

console.log("\nSetelah ditambah:");
console.log(daftarBelanja);

// 5. Menghapus item pertama
let itemDihapus = daftarBelanja.shift();
console.log("\nItem yang dihapus: " + itemDihapus);
```

```
// 6. Mengecek apakah 'Susu' ada di daftar
let adaSusu = daftarBelanja.includes("Susu");
if (adaSusu) {
  console.log("Item 'Susu' ada di dalam daftar belanja.");
} else {
  console.log("Item 'Susu' tidak ada di dalam daftar belanja.");
}

// 7. Menampilkan jumlah item akhir
console.log("\nJumlah item akhir: " + daftarBelanja.length);
```

22.Lalu simpan kan file dan coba jalankan

```
Daftar Belanja:
1. Beras
2. Gula
3. Minyak
4. Telur
5. Mie

Setelah ditambah:
[
  'Beras',  'Gula',
  'Minyak', 'Telur',
  'Mie',    'Susu',
  'Kopi'
]

Item yang dihapus: Beras
Item 'Susu' ada di dalam daftar belanja.
```

23.setelah itu buat kan file 04-array method.js didalam pratikum 3

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main)
$ touch 04 array.method.js
```

24.Kemudian masukan kode nya di bawah ini

```
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];
// --- 1. forEach: menjalankan sesuatu untuk setiap elemen ---
console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
```

```
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});
// --- 2. map: transformasi setiap elemen menjadi nilai baru ---
// Konversi nilai angka ke grade huruf
const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaiMahasiswa);
console.log("Grade :", gradeHuruf);
// --- 3. filter: saring elemen yang memenuhi kondisi ---
const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);
console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaiMahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai gagal :", nilaiGagal);
// --- 4. reduce: mereduksi array menjadi satu nilai ---
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0); // 0 adalah nilai awal akumulator
const rataRata = totalNilai / nilaiMahasiswa.length;
console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai :", totalNilai);
console.log("Rata-rata :", rataRata.toFixed(2));
// --- 5. Menggabungkan beberapa method (method chaining) ---

const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60) // ambil yang lulus dulu
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length; // bagi saat prose
```

25.Simpan kan dan jalan kan di terminal

```
 Mahasiswa-1: 75
 Mahasiswa-2: 90
 Mahasiswa-3: 55
 Mahasiswa-4: 82
 Mahasiswa-5: 68
 Mahasiswa-6: 95
 Mahasiswa-7: 48
 Mahasiswa-8: 77

=== map: Nilai ke Grade ===
Nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Grade : [
  'C', 'A', 'E',
  'B', 'D', 'A',
  'E', 'C'
]

=== filter: Lulus dan Tidak Lulus ===
Semua nilai : [
  75, 90, 55, 82,
  68, 95, 48, 77
]
Nilai lulus : [ 75, 90, 82, 68, 95, 77 ]
Nilai gagal : [ 55, 48 ]
```

26.Setelah itu buatkan latihannya juga analis data produk

```
//latihan-4

// 3. Deklarasi array of objects
const produk = [
  { nama: "Laptop", harga: 8500000, stok: 5 },
  { nama: "Mouse", harga: 150000, stok: 0 },
  { nama: "Keyboard", harga: 450000, stok: 12 },
  { nama: "Monitor", harga: 3200000, stok: 0 },
  { nama: "Headset", harga: 350000, stok: 8 },
];

// 4. Filter produk yang tersedia (stok > 0)
const produkTersedia = produk.filter((item) => item.stok > 0);
console.log("Produk tersedia:", produkTersedia);

// 5. Ambil nama produk dari yang tersedia
const namaProduk = produkTersedia.map((item) => item.nama);
console.log("Nama produk tersedia:", namaProduk);

// 6. Hitung total nilai inventaris (harga * stok)
const totalInventaris = produkTersedia.reduce((total, item) => {
  return total + item.harga * item.stok;
}, 0);
```

```
console.log("Total nilai inventaris: Rp" + totalInventaris.toLocaleString());

// 7. Tampilkan semua produk dengan format tertentu
console.log("\nDaftar Produk:");
produk.forEach((item) => {
  if (item.stok > 0) {
    console.log(
      `[TERSEDIA] ${item.nama} - Rp${item.harga.toLocaleString()} (${item.stok}
unit)`,
    );
  } else {
    console.log(`[HABIS] ${item.nama} - Rp${item.harga.toLocaleString()}`);
  }
});
```

27.Simpankan kode nya dulu baru jalan kan

```
Produk tersedia: [
  { nama: 'Laptop', harga: 8500000, stok: 5 },
  { nama: 'Keyboard', harga: 450000, stok: 12 },
  { nama: 'Headset', harga: 350000, stok: 8 }
]
Nama produk tersedia: [ 'Laptop', 'Keyboard', 'Headset'
]
Total nilai inventaris: Rp50.700.000

Daftar Produk:
[TERSEDIA] Laptop - Rp8.500.000 (5 unit)
[HABIS] Mouse - Rp150.000
[TERSEDIA] Keyboard - Rp450.000 (12 unit)
[HABIS] Monitor - Rp3.200.000
[TERSEDIA] Headset - Rp350.000 (8 unit)
```

28.Selanjutnya kita buatkan file baru di modul pratikum-3 yang Bernama touch 05 rekursi

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main)
$ touch 05 rekursi.js
```

29.Kemudian masukan kodenya yang di file touch 05 rekursi

```
function faktorial(n) {
  // Base case: berhenti saat n = 1
  if (n <= 1) return 1;
  // Recursive case: n × faktorial(n-1)
  return n * faktorial(n - 1);
}
console.log("=== Faktorial ===");
console.log("0! =", faktorial(0)); // 1 (konvensi matematika)
console.log("1! =", faktorial(1)); // 1
console.log("5! =", faktorial(5)); // 120
console.log("7! =", faktorial(7)); // 5040

// --- 2. Fibonacci: F(n) = F(n-1) + F(n-2) ---
// Deret: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...
function fibonacci(n) {
  // Base case
  if (n === 0) return 0;
  if (n === 1) return 1;
  // Recursive case
  return fibonacci(n - 1) + fibonacci(n - 2);
}
console.log("\n=== Fibonacci ===");
for (let i = 0; i <= 8; i++) {
  process.stdout.write(fibonacci(i) + " ");
}
console.log(""); // pindah baris
// --- 3. Rekursi untuk menelusuri array ---
function jumlahArray(arr, indeks = 0) {
  // Base case: sudah sampai akhir array
  if (indeks === arr.length) return 0;
  // Recursive case: elemen saat ini + jumlah sisa array
  return arr[indeks] + jumlahArray(arr, indeks + 1);
}
const angka = [3, 7, 2, 9, 5];
console.log("\n=== Jumlah Array dengan Rekursi ===");
console.log("Array :", angka);
console.log("Jumlah :", jumlahArray(angka)); // 26
// --- 4. Countdown rekursif ---
function countdown(n) {
  if (n < 0) {
    console.log("Selesai!");
    return;
```

30.Simpankan dulu lalu jalankan

```
$ node 05-rekursi.js
=== Faktorial ===
0! = 1
1! = 1
5! = 120
7! = 5040

=== Fibonacci ===
0 1 1 2 3 5 8 13 21

=== Jumlah Array dengan Rekursi ===
Array : [ 3, 7, 2, 9, 5 ]
Jumlah : 26

=== Countdown Rekursif ===
5
4
3
2
1
0
Selesai!
```

31.Setelah semua selesai lalu buat kan file baru yang Bernama tugas pratikum 3 didalam

Modul 3

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main)
$ mkdir tugas
```

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data (main)
$ touch tugas 01.js tugas 02.js
```

Tugas 1:Sistem nilai Mahasiswa

```
const dataMahasiswa = [
  { nama: "Andi", nilai: 85 },
  { nama: "Budi", nilai: 70 },
  { nama: "Citra", nilai: 90 },
  { nama: "Dewi", nilai: 60 },
  { nama: "Eka", nilai: 75 },
  { nama: "Fajar", nilai: 50 },
];

// 3. Function hitungStatistik (pakai reduce)
function hitungStatistik(arrMahasiswa) {
```

```
const hasil = arrMahasiswa.reduce(
  (acc, mhs, index) => {
    acc.total += 1;
    acc.jumlahNilai += mhs.nilai;

    if (mhs.nilai > acc.tertinggi) acc.tertinggi = mhs.nilai;
    if (mhs.nilai < acc.terendah) acc.terendah = mhs.nilai;

    return acc;
  },
  {
    total: 0,
    jumlahNilai: 0,
    tertinggi: -Infinity,
    terendah: Infinity,
  },
);

return {
  total: hasil.total,
  rataRata: (hasil.jumlahNilai / hasil.total).toFixed(2),
  tertinggi: hasil.tertinggi,
```

32.Simpan lalu jalankan

```
=== Statistik Mahasiswa ===
Total       : 6
Rata-rata   : 71.67
Tertinggi   : 90
Terendah    : 50

=== Mahasiswa Lulus (>= 70) ===
1. Andi - 85
2. Budi - 70
3. Citra - 90
4. Eka - 75

=== Data Mahasiswa + Grade ===
1. Andi - 85 (Grade: A)
2. Budi - 70 (Grade: C)
3. Citra - 90 (Grade: A)
4. Dewi - 60 (Grade: D)
5. Eka - 75 (Grade: B)
6. Fajar - 50 (Grade: D)
```

33.Setelah itu buatkan tugas 2 yang Bernama pangkat dan palingdrom rekursif

```
// 2. Function rekursif pangkat
function pangkat(basis, eksponen) {
  // base case
  if (eksponen === 0) return 1;

  // recursive case
  return basis * pangkat(basis, eksponen - 1);
}

// Uji function pangkat
console.log("=== Uji Fungsi Pangkat ===");
console.log(`2^3 = ${pangkat(2, 3)}`);
console.log(`5^2 = ${pangkat(5, 2)}`);
console.log(`7^0 = ${pangkat(7, 0)}`);

// 3. Function rekursif balikString
function balikString(str) {
  // base case
  if (str.length <= 1) return str;

  // recursive case
  return str[str.length - 1] + balikString(str.slice(0, str.length - 1));
}

// Uji balikString
console.log("\n=== Uji Balik String ===");
console.log(`halo -> ${balikString("halo")}`);
console.log(`javascript -> ${balikString("javascript")}`);

// 4. Function cekPalindrom
function cekPalindrom(str) {
  const hasilBalik = balikString(str);
  return str === hasilBalik;
}

// Uji palindrom
console.log("\n=== Uji Palindrom ===");
const kataUji = ["katak", "civic", "level", "buku"];

kataUji.forEach((kata, index) => {
  const hasil = cekPalindrom(kata) ? "Palindrom" : "Bukan Palindrom";
  console.log(`${index + 1}. ${kata} -> ${hasil}`);
});
```

34.simpan dulu lalu coba jalankan diterminal

```
LENOVO@LAPTOP-89DOR1F4 MINGW64 ~/2025573010054_Algoritma_Dan_Struktur_Data/pratikum-3/tugas (main)
$ node tugas-2.js
=== Uji Fungsi Pangkat ===
2^3 = 8
5^2 = 25
7^0 = 1

=== Uji Balik String ===
halo -> olah
javascript -> tpircsavaj

=== Uji Palindrom ===
1. katak -> Palindrom
2. civic -> Palindrom
3. level -> Palindrom
4. buku -> Bukan Palindrom
```

BAB IV
PENUTUP

4.1 Kesimpulan

Function dan array merupakan dua konsep dasar dalam pemrograman yang memiliki peran penting dalam pengembangan aplikasi. Function adalah kumpulan perintah yang dibuat untuk menjalankan tugas tertentu dan dapat digunakan kembali tanpa harus menulis ulang kode yang sama. Dengan adanya function, struktur program menjadi lebih rapi, mudah dipahami, serta lebih efisien dalam proses penulisan kode. Function juga dapat menerima input berupa parameter dan menghasilkan output, sehingga penggunaannya menjadi lebih fleksibel sesuai kebutuhan.

Selain itu, array adalah struktur data yang digunakan untuk menyimpan sekumpulan nilai dalam satu variabel dengan tipe data yang sama. Setiap data dalam array memiliki indeks yang memudahkan dalam proses pengaksesan dan pengolahan. Penggunaan array membantu programmer dalam mengelola data dalam jumlah banyak tanpa harus membuat banyak variabel, sehingga lebih praktis dan terorganisir.

Dalam penerapannya, function dan array sering digunakan secara bersamaan. Function dapat dimanfaatkan untuk mengolah data yang tersimpan di dalam array, seperti melakukan perhitungan, pencarian, maupun pengolahan data lainnya. Kombinasi keduanya membuat program menjadi lebih terstruktur, mudah dikembangkan, serta meningkatkan efisiensi dalam penulisan kode. Selain itu, penggunaan function dan array juga mempermudah proses perbaikan kesalahan (debugging) karena kode menjadi lebih jelas dan terorganisir.